

e automated tooling, in some cases also users visiting web interface

- Web Runners API: API calls from the Runners, that allows Runner to fetch new jobs, or update trace log
- Web GraphQL: provide a flexible API interface, allowing the Web frontend to fetch only the data needed thereby reducing the amount of compute and data transfer
- Web ActionCable: provide bi-directional connection to enable real-time features for Users visiting web interface
- Web Feature Flags Unleash Backend: provide an Unleash-compatible Server that uses GitLab API
- Web Packages API: provide a REST API compatible with the packaging tools: Debian, Maven, container registry proxy, etc.
- Git nodes: all code required to authorize git pull/push over SSH or HTTPS
- Sidekiq: run background jobs
- Services/Models/DB: all code required to maintain our database structure, data validation, business logic, and policies models that needs to be shared with other components

The best way to likely describe how the actual GitLab Rails split would look like. It is a satellite model.

Where we have a single core, that is shared across all satellite components. The design of that implies

that satellite components have a limited way to communicate with each other. In a single monolithic application

in most of cases application would communicate with a code. In a satellite model the communication needs

to be performed externally to the component. This can be via Database, Redis or using a well defined exposed API.

mermaid

flowchart TD

subgraph Data Store

D[Database]

R[Redis]

end

subgraph Rails Engines

subgraph Data Access Layer

C[Core]

end

subgraph Web Processing

W[Web]

end

subgraph Background Processing

S[Sidekiq]

end

end

C --> D & R

W & S -- using application models --> C

R -- push background job --> S

W -- via async schedule --> S

S -- via Web API --> W

Application Layers for on-premise installations

The on-premise installations are significantly smaller and they usually run GitLab Rails in two main flavors:

mermaid

graph LR

gitlabnode[GitLab Node with Load Balancer]

gitlabnodeweb[Web running Puma]

gitlabnodesidekiq[Background jobs running Sidekiq]

gitlabnodegit[Git running Puma and SSH]

subgraph GitLab Rails

gitlabrailswebcontrollers[Controllers]

gitlabrailsapi[API]

gitlabrailsapirunners[API Runner]

gitlabrailsgraphql[GraphQL]

gitlabrailsactioncable[ActionCable]

gitlabrailsservices[Services]

gitlabrailsmodels[Models]

gitlabrailssidekiq[Sidekiq Workers]

end

postgresqldb[(PostgreSQL Database)]

redisdb[(Redis Database)]

gitlabnode --> gitlabnodeweb

gitlabnode --> gitlabnodesidekiq

gitlabnode --> gitlabnodegit

gitlabnodeweb --> gitlabrailswebcontrollers

gitlabnodeweb --> gitlabrailsapi

gitlabnodeweb --> gitlabrailsapirunners

gitlabnodeweb --> gitlabrailsgraphql

gitlabnodeweb --> gitlabrailsactioncable

gitlabnodegit --> gitlabrailsapi

gitlabnodesidekiq --> gitlabrailssidekiq

gitlabrailswebcontrollers --> gitlabrailsservices

gitlabrailsapi --> gitlabrailsservices

gitlabrailsapirunners --> gitlabrailsservices

gitlabrailsgraphql --> gitlabrailsservices

gitlabrailsactioncable --> gitlabrailsservices

gitlabrailssidekiq --> gitlabrailsservices

gitlabrailsservices --> gitlabrailsmodels

gitlabrailsmodels --> postgresqldb

gitlabrailsmodels --> redisdb

Application Layers on GitLab.com

Due to its scale, GitLab.com requires much more attention to run. This is needed to better manage resources

and provide SLAs for different functional parts. The chart below provides a simplistic view of GitLab.com application layers.

It does not include all components, like Object Storage nor Gitaly nodes, but shows the GitLab Rails dependencies between different components and how they are configured on GitLab.com today:

mermaid

graph LR

gitlabcomlb[GitLab.com Load Balancer]

gitlabcomweb[Web Nodes running Puma]

gitlabcomapi[API Nodes running Puma]

gitlabcomwebsockets[WebSockets Nodes running Puma]

gitlabcomsidekiq[Background Jobs running Sidekiq]

gitlabcomgit[Git Nodes running Puma and SSH]

subgraph GitLab Rails

gitlabrailswebcontrollers[Controllers]

gitlabrailsapi[API]

gitlabrailsapirunners[API Runner]

gitlabrailsgraphql[GraphQL]

gitlabrailsactioncable[ActionCable]

gitlabrailsservices[Services]

gitlabrailsmodels[Models]

gitlabrailssidekiq[Sidekiq Workers]

end

postgresqldb[(PostgreSQL Database)]

redisdb[(Redis Database)]

gitlabcomlb --> gitlabcomweb

gitlabcomlb --> gitlabcomapi

gitlabcomlb --> gitlabcomwebsockets

gitlabcomlb --> gitlabcomgit

gitlabcomweb --> gitlabrailswebcontrollers

gitlabcomapi --> gitlabrailsapi

gitlabcomapi --> gitlabrailsapirunners

gitlabcomapi --> gitlabrailsgraphql

gitlabcomwebsockets --> gitlabrailsactioncable

gitlabcomgit --> gitlabrailsapi

gitlabcomsidekiq --> gitlabrailssidekiq

gitlabrailswebcontrollers --> gitlabrailsservices

gitlabrailsapi --> gitlabrailsservices

gitlabrailsapirunners --> gitlabrailsservices
gitlabrailsgraphql --> gitlabrailsservices
gitlabrailsactioncable --> gitlabrailsservices
gitlabrailssidekiq --> gitlabrailsservices

gitlabrailsservices --> gitlabrailsmodels

gitlabrailsmodels --> postgresqldb
gitlabrailsmodels --> redisdb

Layer dependencies

The differences in how GitLab is run for on-premise versus how we run GitLab.com does show a main division line in GitLab Rails:

- Web: containing all API, all Controllers, all GraphQL and ActionCable functionality
- Sidekiq: containing all background processing jobs
- Core: containing all database, models and services that needs to be shared between Web and Sidekiq

Each of these top-level application layers do depend only on a fraction of the codebase with all relevant dependencies:

- In all cases we need the underlying database structure and application models
- In some cases we need dependent services
- We only need a part of the application common library
- We need gems to support the requested functionality
- Individual layers should not use another sibling layer (tight coupling), rather connect via API, Redis or DB to share data (loose coupling)

Proposal

The Memory team group conducted a Proof-of-Concept phase to understand the impact of introducing Application Layers.

We did this to understand the complexity, impact, and needed iterations to execute this proposal.

The proposals here should be treated as evaluation of the impact of this blueprint, but not a final solution to be implemented. The PoC as defined is not something that should be merged, but serves as a basis for future work.

PoC using Rails Engines

We decided to use Rails Engines by modeling a Web Application Layer. The Web Engine contained Controllers, API, GraphQL.

This allowed us to run Web Nodes with all dependencies, but measure the impact on Sidekiq not having these components loaded.

All work can be found in these merge requests:

- Provide mechanism to load GraphQL with all dependencies only when needed
- Draft: PoC - Move GraphQL to the WebEngine
- Draft: PoC - Move Controllers and Grape API:API to the WebEngine
- Draft: PoC - Move only Grape API:API to the WebEngine
- Measure performance impact for proposed webengine

What was done?

- We used Rails Engines
- The 99% of changes as visible in the above MRs is moving files as-is
- We moved all GraphQL code and specs into engines/webengine/ as-is
- We moved all API and Controllers code and specs into engines/webengine
- We adapted CI to test engines/webengine/ as a self-sufficient component of stack
- We configured GitLab to load gem webengine running Web nodes (Puma web server)
- We disabled loading webengine when running Background processing nodes (Sidekiq)

Implementation details for proposed solution

1. Introduce new Rails Engine for each application layer.

We created engines folder, which could contain different engines for each application layer we introduce in the future.

In the above PoCs we introduced the new Web Application Layer, located in engines/webengine folder.

1. Move all code and specs into engines/webengine/

- We moved all GraphQL code and specs into engines/webengine/ without changing files itself
- We moved all Grape API and Controllers code into engines/webengine/ without changing files itself

1. Move gems to the engines/webengine/

- We moved all GraphQL gems to the actual webengine Gemfile
- We moved Grape API gem to the actual webengine Gemfile

ruby

```
Gem::Specification.new do |spec|
  spec.adddependency 'apollouploadserver'
  spec.adddependency 'graphql'
  spec.adddependency 'graphiql-rails'

  spec.adddependency 'graphql-docs'
  spec.adddependency 'grape'
end
```

1. Move routes to the engines/webengine/config/routes.rb file

- We moved GraphQL routes to the webengine routes.
- We moved API routes to the webengine routes.
- We moved most of the controller routes to the webengine routes.

```
ruby
Rails.application.routes.draw do
  post '/api/graphql', to: 'graphql#execute'
  mount GraphQL::Rails::Engine, at: '/-graphql-explorer', graphqlpath:
  Gitlab::Utils.appendpath(Gitlab.config.gitlab.relative_url_root, '/api/graphql')

  draw :api

  ...
end
```

1. Move initializers to the engines/webengine/config/initializers folder

- We moved graphql.rb initializer to the webengine initializers folder
- We moved grapepatch.rb and grapevalidators to the webengine initializers folder

1. Connect GitLab application with the WebEngine

In GitLab Gemfile.rb, add webengine to the engines group

```
ruby
Gemfile
group :engines, :test do
  gem 'webengine', path: 'engines/webengine'
end
```

Since the gem is inside :engines group, it is not automatically required by default.

1. Configure GitLab when to load the engine.

In GitLab config/engines.rb, we can configure when do we want to load our engines by relying on our Gitlab::Runtime

```
ruby
config/engines.rb
Load only in case we are running webserver or rails console
if Gitlab::Runtime.puma? || Gitlab::Runtime.console?
  require 'webengine'
end
```

1. Configure Engine

Our Engine inherits from the Rails::Engine class. This way this gem notifies Rails that there's an engine at the specified path so it will correctly mount the engine inside the application, performing tasks such as adding the app directory of the engine to the load path for models, mailers, controllers, and views.

A file at lib/webengine/engine.rb, is identical in function to a standard Rails application's config/application.rb file. This way engines can access a configuration object which contains configuration shared by all railties and the application.

Additionally, each engine can access autoloadpaths, eagerloadpaths, and autoloadoncepaths settings which are scoped to that engine.

```
ruby
module WebEngine
  class Engine < ::Rails::Engine
    config.eagerloadpaths.push(%W[{{config.root}}/lib
                                {{config.root}}/app/graphql/resolvers/concerns
                                {{config.root}}/app/graphql/mutations/concerns
                                {{config.root}}/app/graphql/types/concerns])

    if Gitlab.ee?
      eepaths = config.eagerloadpaths.eachwithobject([]) do |path, memo|
        eepath = config.root
          .join('ee', Pathname.new(path).relativepathfrom(config.root))
        memo << eepath.to_s
      end
      Eager load should load CE first
      config.eagerloadpaths.push(eepaths)
    end
  end
end
```

1. Testing

We adapted CI to test engines/webengine/ as a self-sufficient component of stack.

- We moved spec as-is files to the engines/webengine/spec folder
- We moved ee/spec as-is files to the engines/webengine/ee/spec folder
- We control specs from main application using environment variable TESTWEBENGINE
- We added new CI job that will run engines/webengine/spec tests separately using TESTWEBENGINE environment variable.
- We added new CI job that will run engines/webengine/ee/spec tests separately using TESTWEBENGINE environment variable.
- We are running all white box frontend tests with TESTWEBENGINE=true

Results

The effect on introducing these changes:

- Savings for RSS
- 61.06 MB (7.76%) - Sidekiq without GraphQL

- 100.11 MB (12.73%) - Sidekiq without GraphQL and API
- 208.83 MB (26.56%) - Sidekiq without GraphQL, API, Controllers
- The size of Web nodes (running Puma) stayed the same as before

Savings on Sidekiq start-up event, for a single Sidekiq cluster without GraphQL, API, Controllers

- We saved 264.13 MB RSS (28.69%)
- We saved 264.09 MB USS (29.36%)
- Boot-up time was reduced from 45.31 to 21.80 seconds. It was 23.51 seconds faster (51.89%)
- We have 805,772 less live objects, 4,587,535 less allocated objects, 2,866 less allocated pages and 3.65 MB less allocated space for objects outside of the heap
- We loaded 2,326 less code files (15.64%)
- We reduced the duration of a single full GC cycle from 0.80 seconds to 0.70 seconds (12.64%)

Puma single, showed very little difference as expected.

More details can be found in the issue.

Impact on GitLab.com

Estimating the results for the scale of running GitLab.com, today we use:

- Individual GC cycle takes around 130 ms for Web and 200 ms for Sidekiq on GitLab.com
- On average we do around 2 GC cycles per-second or 0.12 cycles per second for Sidekiq
- This translates to using around 9.5 vCPUs per-second for Web and around 8 vCPUs per-second for Sidekiq of spend on GC alone
- Sidekiq uses 2.1 GB on average or 550 GB in total of memory on GitLab.com

We estimate the possible maximum savings for introducing webengine:

- Reduce a GC cycle time by 20%, from to 200 ms to 160 ms
- The amount of GC cycles per-second would stay the same, but due to GC cycle time reduction we would use around 6 vCPUs instead of 8 vCPUs
- In the best case we would be looking at Sidekiq alone we would be estimating to save up-to 137 GB of memory on GitLab.com

This model could be extended to introduce sidekiqengine giving a similar benefits (even more important due to visible impact on users) for Web nodes.

Outcome

We achieved a number of benefits introducing these changes.

Pros:

- Significantly lower memory usage

- Significantly shorter application load time for Sidekiq
- Significantly improved responsiveness of Sidekiq service due to much shorter GC cycles
- Significantly easier testing of a portion of application, ex. changing webengines/ does require re-running test only for this application layer
- We retained a monolithic architecture of the codebase, but sharing database and application models
- A significant saving from the infrastructure side
- Ability to comfortably run on constrained environments by reducing application footprint

Cons:

- It is harder to implement GraphQL subscriptions as in case of Sidekiq as we need another way to pass subscriptions
- apiv4 paths can be used in some services that are used by Sidekiq (for example `apiv4projectspath`)
- urlhelpers paths are used in models and services, that could be used by Sidekiq (for example `Gitlab::Routing.urlhelpers.projectpipelinespath` is used by `ExpirePipelineCacheService` in `ExpirePipelineCacheWorker`)

Example: GraphQL

Draft: PoC - Move GraphQL to the WebEngine

- The 99% of changes as visible in the above MRs is moving files as-is.
- The actual work on fixing cross-dependencies, specs, and configuring webengine
- We adapted CI to test engines/webengine/ as a self-sufficient component of stack

Today, loading GraphQL requires a bunch of dependencies:

- > We also discovered that we load/require 14480 files, `memory-team-2gb-week9`
- > when we start GitLab. 1274 files belong to GraphQL. This means that if we don't load 1274 application files
- > and all related GraphQL gems when we don't need them (Sidekiq), we could save a lot of memory.

GraphQL only needs to run in a specific context. If we could limit when it is being loaded we could effectively improve application efficiency, by reducing application load time and required memory. This, for example, is applicable for every size installation.

A potential challenge with GraphQL and Websockets is that at some point we might want to use Action Cable subscriptions and push GraphQL/API payload from Sidekiq to clients. This would likely use Redis to pass data through. Where Sidekiq would publish information on Redis and ActionCable Node would pass through that information to connected clients. This way of working is possible in the above model, but we would have to use GraphQL or API (over HTTP endpoint) to calculate what should be sent.

An alternative way is to use a notification system that would always make an ActionCable node (the one handling WebSockets) generate a payload based on a send query instead of performing passthrough. This could be applicable since ActionCable is the one handling a given connection

for a client. This could have a downside of having to recalculate the same payload if many clients would be watching the same resource. However, this behavior of system might still be desired for security purposes, as generated payload might be dependent on permission of watching client (we would show different for anonymous, and different for the member of the project).

Example: API

Draft: PoC - Move only Grape API:API to the WebEngine

- 99% of the changes, as visible in the above MRs, are moving the files as-is.
- The actual work on fixing cross-dependencies, specs, configuring initializers, gems and routes.

Grape::API is another example that only needs to run only in a web server context.

Potential challenges with Grape API:

- Currently there are some API::API dependencies in the models (for example API::Helpers::Version dependency in project model or API::API dependency in GeoNode model for georetrieveurl)
- apiv4 paths are used in helpers, presenters, and views (for example apiv4projectspath in PackagesHelper)

Example: Controllers

Draft: PoC - Move Controllers and Grape API:API to the WebEngine

- 99% of the changes, as visible in the above MRs, are moving files as-is.
- The actual work on fixing cross-dependencies, specs, configuring initializers, gems and routes.

Controllers, Serializers, some presenters and some of the Grape:Entities are also good examples that only need to be run in web server context.

Potential challenges with moving Controllers:

- We needed to extend Gitlab::Patch::DrawRoute to support engines/webengine/config/routes and engines/webengine/ee/config/routes in case when webengine is loaded. Here is potential solution.
- Gitlab::Routing.urlhelpers paths are used in models and services, that could be used by Sidekiq (for example Gitlab::Routing.urlhelpers.projectpipelinespath is used by ExpirePipelineCacheService in ExpirePipelineCacheWorker))

Packwerk

NOTE:

Packwerk is currently accepting bug fixes only, and it is not being actively developed. Check for more details

Future impact

Application Layers and this proposal currently defines only webengine. Following the same pattern we could easily introduce additional engines dedicated for supporting that would allow us to maintain much better separation, lower memory usage and much better maintainability of GitLab Rails into the future.

This would be a framework for introducing all new interfaces for features that do not need to be part of the core codebase, like support for additional Package services. Allowing us to better scale application in the future, but retaining a single codebase and monolithic architecture of GitLab.

As of today, it seems reasonable to define three application layers:

- gitlab-core: a core functionality: DB structure, models, services, common library. It models a data access layer, and initially all services needed to run GitLab. This might be potentially be split in the future into smaller aspects
- gitlab-web: a Controllers/API/GraphQL/ActionCable functionality needed to run in a web server context (depends on gitlab-core)
- gitlab-sidekiq: a background jobs functionality needed to run Sidekiq Workers (depends on gitlab-core)

This model is best described today as a shared core with satellite. The shared core defines data access layer, where as satellites define a way to present and process this data. Satellites can only talk with Core. They cannot directly load or talk to another satellite unless they use a well defined interface in form of API, GraphQL or Redis (as for scheduling Sidekiq jobs).

It is reasonable to assume that we limit how many engines we allow. Initial proposal is to allow up to 5 engines to be created to ensure that we do not have explosion of engines.

Issues and Merge Requests

- Split application into functional parts to ensure that only needed code is loaded with all dependencies
- Provide mechanism to load GraphQL with all dependencies only when needed
- Draft: PoC - Move GraphQL to the WebEngine
- Draft: PoC - Move Controllers and Grape API:API to the WebEngine
- Draft: PoC - Move only Grape API:API to the WebEngine
- Measure performance impact for proposed webengine
- Create new models / classes within a module / namespace
- Make teams to be maintainers of their code
- Use nested structure to organize CI classes
- WIP: Make it simple to build and use "Decoupled Services"
- Rails takes awhile to boot, let's see if we can improve this

SECTION: engineering/architecture/design-
documents/consolidating_groups_and_projects/_index.md

{{< engineering/design-document-header >}}

Numerous features exist exclusively within groups or projects. The boundary between group and project features used to be clear.

However, there is growing demand to have group features in projects, and project features in groups. For example, having issues in groups, and epics in projects.

The Simplify Groups & Projects Working Group determined that our architecture is a significant hurdle in sharing features across groups and projects.

Architecture issue: <<https://gitlab.com/gitlab-org/architecture/tasks/-/issues/7>>

Challenges

Feature duplication

When a feature needs to be made available on a different level, we have no established process in place. This results in the reimplementing of the same feature. Those implementations diverge from each other over time as they all live on their own. A few more problems with this approach:

- Features are coupled to their container. In practice, it is not straight forward to decouple a feature from its container. The degree of coupling varies across features.
- Naive duplication of features will result in a more complex and fragile codebase.
- Generalizing solutions across groups and projects may degrade system performance.
- The range of features spans across many teams, and these changes will need to manage development interference.
- The group/project hierarchy creates a natural feature hierarchy. When features exist across containers the feature hierarchy becomes ambiguous.
- Duplication of features slows down development velocity.

There is potential for significant architectural changes. These changes will have to be independent of the product design, so that customer experience remains consistent.

Performance

Resources can only be queried in elaborate/complicated ways. This caused performance issues with authorization, epics, and many other places. As an example, to query the projects a user has access to, the following sources need to be considered:

- Personal projects

- Direct group membership
- Direct project membership
- Inherited group membership
- Inherited project membership
- Group sharing
- Inherited membership via group sharing
- Project sharing

Group/project membership, group/project sharing are also examples of duplicated features.

Goals

For now, this blueprint strictly relates to the engineering challenges.

- Consolidate the group and project container architecture.
- Develop a set of solutions to decouple features from their container.
- Decouple engineering changes from product changes.
- Develop a strategy to make architectural changes without adversely affecting other teams.
- Provide a solution for requests asking for features to be made available at other levels.

Proposal

Use our existing Namespace model as a container for features. We already have a Namespace associated with User (personal namespace), and with Group (which is a subclass of Namespace). We can extend this further, by associating Namespace with Projects by introducing ProjectNamespaces. Each Project should be owned by its ProjectNamespace, and this relation should replace the existing Project <-> Group / personal namespace relation.

We also lack a model specific for personal namespaces, and we use the generic Namespace model instead. This is confusing, but can be fixed by creating a dedicated subclass: UserNamespace.

As a result, the Namespace hierarchy will transition to:

```
mermaid
classDiagram
    Namespace <|-- UserNamespace
    Namespace <|-- Group
    Namespace <|-- ProjectNamespace
```

New features should be implemented on Namespace. Similarly, when a feature need to be reimplemented on a different level, moving it to Namespace essentially makes it available on all levels:

- Personal namespaces
- Groups

- Projects

Various traversal queries are already available on Namespaces to query the group hierarchy. Projects represent the leaf nodes in the hierarchy, but with the introduction of ProjectNamespace, these traversal queries can be used to retrieve projects as well.

This also enables further simplification of some of our core features:

- Routes should be generated based on the Namespace hierarchy, instead of mixing the project with the group hierarchy.
- There is no need to differentiate between GroupMembers and ProjectMembers. All Members should be related to a Namespace. This can lead to simplified querying, and potentially deduplicating policies.

As more and more features will be migrated to Namespace, the role of the Project model will diminish over time to essentially a container around the repository related functionality.

Iterations

The work required to establish Namespace as a container for our features is tracked under Consolidate Groups and Projects epic.

Phase 1 (complete)

- Phase 1 epic.
- Goals:
 1. Ensure every project receives a corresponding record in the namespaces table with type='Project'.
 1. For user namespaces, the type changes from NULL to User.

We should make sure that projects, and the project namespace, are equivalent:

- Create project: Use Rails callbacks to ensure a new project namespace is created for each project. Project namespace records should contain createdat and updatedat attributes equal to the project's createdat/updatedat attributes.
- Update project: Use the aftersave callback in Rails to ensure some attributes are kept in sync between project and project namespaces.
Read projectaftersave for more information.
- Delete project: Use FKs cascade delete or Rails callbacks to ensure when a Project or its ProjectNamespace is removed, its corresponding ProjectNamespace or Project is also removed.
- Transfer project to a different group: Make sure that when a project is transferred, its corresponding project namespace is transferred to the same group.
- Transfer group: Make sure when transferring a group that all of its sub-projects, either direct or through descendant groups, have their corresponding project namespaces transferred correctly as well.

- Export or import project
 - Export project continues to export only the project, and not its project namespace, in this phase. The project namespace does not contain any specific information to export at this point. Eventually, we want the project namespace to be exported as well.
 - Import project creates a new project, so the project namespace is created through Rails aftersave callback on the project model.
- Export or import group: When importing or exporting a Group, projects are not included in the operation. If that feature is changed to include Project when its group is imported or exported, the logic must include their corresponding project namespaces in the import or export.

After ensuring these points, run a database migration to create a ProjectNamespace record for every Project. Project namespace records created during the migration should have createdat and updatedat attributes set to the migration runtime. The project namespaces' createdat and updatedat attributes would not match their corresponding project's createdat and updatedat attributes. We want the different dates to help audit any of the created project namespaces, in case we need it. After this work completes, we must migrate data as described in Backfill ProjectNamespace for every Project.

Phase 2 (complete)

- Phase 2 epic.
- Goal: Link ProjectNamespace to other entities on the database level.

In this phase:

- Communicate the changes company-wide at the engineering level. We want to make engineers aware of the upcoming changes, even though teams are not expected to collaborate actively until phase 3.
- Raise awareness to avoid regressions and conflicting or duplicate work that can be dealt with before phase 3.

Phase 3 (ongoing)

- Phase 3 epic.

In this phase we are migrating basic, high-priority project functionality from Project to ProjectNamespace, or directly to Namespace. Problems to solve as part of this phase:

- Unify members/members actions - on UI and API level.
- Starring: Right now only projects can be starred. We want to bring this to the group level.
- Common actions: Destroying, transferring, restoring. This can be unified on the controller level and then propagated lower.
- Archiving currently only works on the project level. This can be brought to the group level, similar to the mechanism for "pending deletion".
- Avatar's serving and actions.

Phase 4

- Phase 4 epic

In this phase we are migrating additional functionality from Project to ProjectNamespace/Namespace:

- Replace usages of Project with ProjectNamespace in the code.
- API changes to expose namespaces and namespace features.
 - Investigate if we extend API for groups or we introduce a namespaces endpoint and slowly deprecate groups and projects endpoints.
- Break down each feature that needs to be migrated from Project to ProjectNamespace or Namespace.
 - Investigate if we can move a feature from Project -> Namespace directly vs Project -> ProjectNamespace -> Namespace. This can be decided on a feature by feature case.
- Migrate Projectnamespace to reference ProjectNamespace.
- Routes consolidation between Project & ProjectNamespace.
- Policies consolidation.

Phase 5

- Phase 5 epic

We should strive to do the code clean up as we move through the phases. However, not everything can be cleaned up while something is still being developed. For example, dropping database columns can be done as the last task when we are sure everything is working. This phase will focus on:

- Code cleanup
- Database cleanup

Migrating features to Namespaces

The initial iteration will provide a framework to house features under Namespaces. Stage groups will eventually need to migrate their own features and functionality over to Namespaces. This may impact these features in unexpected ways. Therefore, to minimize UX debt and maintain product consistency, stage groups will have to consider several factors when migrating their features over to Namespaces:

1. Conceptual model: What are the current and future state conceptual models of these features (see object modeling for designers)? These should be documented in Pajamas (example: merge requests).
1. Merge conflicts: What inconsistencies are there across project, group, and administrator levels? How might these be addressed? For an example of how we rationalized this for labels, see this issue.
1. Inheritance & information flow: How is information inherited across our container hierarchy currently? How might this be impacted if complying with the new inheritance behavior framework?
1. Settings: Where can settings for this feature be found currently? How will these be impacted by Namespaces?
1. Access: Who can access this feature and is that impacted by the new container structure? Are there any role or privacy considerations?
1. Tier: Is there any tier functionality that is differentiated by projects and groups?

1. Documentation: Is the structure and content of documentation impacted by these changes at all?

1. Solution proposal:

- Think big: This analysis provides a great opportunity to zoom out and consider the feature UX as a whole. How could you make this feature lovable based on the new structure, inheritance, and capabilities afforded by Namespaces? Is there any UI which doesn't comply with Pajamas?

- Start small: What are the product changes that need to be made to assist with the migration?

- Move fast: Prioritise these solution ideas, document in issues, and create a roadmap for implementation.

Related topics

- Organization developer documentation

- Organization user documentation

SECTION: engineering/architecture/design-documents/container_registry_metadata_database/_index.md

{{< engineering/design-document-header >}}

Usage of the GitLab container registry

With the container registry integrated into GitLab, every GitLab project can have its own space to store its Docker images. You can use the registry to build, push and share images using the Docker client, CI/CD or the GitLab API.

Each day on GitLab.com, between 150k and 200k images are pushed to the registry, generating about 700k API events. It's also worth noting that although some customers use other registry vendors, more than 96% of instances are using the GitLab container registry.

For GitLab.com and for GitLab customers, the container registry is a critical component to building and deploying software.

Current Architecture

The container registry is a single Go application. Its only dependency is the storage backend on which images and metadata are stored.

mermaid

graph LR

C((Client)) -- HTTP request --> R(Container Registry) -- Upload/download blobs --> B(Storage Backend)

R -- Write/read metadata --> B

Client applications (for example, GitLab Rails and Docker CLI) interact with the container registry through its HTTP API. The most common operations are pushing and pulling images

to/from the registry, which require a series of HTTP requests in a specific order. The request flow for these operations is detailed in the Request flow.

The registry supports multiple storage backends, including Google Cloud Storage (GCS) which is used for the GitLab.com registry. In the storage backend, images are stored as blobs, deduplicated, and shared across repositories. These are then linked (like a symlink) to each repository that relies on them, giving them access to the central storage location.

The name and hierarchy of repositories, as well as image manifests and tags are also stored in the storage backend, represented by a nested structure of folders and files. This video gives a practical overview of the registry storage structure.

Clients

The container registry has two main clients: the GitLab Rails application and the Docker client/CLI.

Docker

The Docker client (docker CLI) interacts with the GitLab container registry mainly using the login, push, and pull commands.

Login and Authentication

GitLab Rails is the default token-based authentication provider for the GitLab container registry.

Once the registry receives a request sent by an unauthenticated Docker client, it will reply with 401 Unauthorized and instruct the client to obtain a token from the GitLab Rails API. The Docker client will then request a Bearer token and embed it in the Authorization header of all requests. The registry is responsible for determining if the user is authentication/authorized to perform those requests based on the provided token.

mermaid

sequenceDiagram

participant C as Docker client

participant R as GitLab container registry

participant G as GitLab Rails

C->>R: docker login gitlab.example.com

R->>C: 401 Unauthorized

Note left of R: The response includes the realm (for example, <https://gitlab.example.com/jwt/auth>)
 from where a token should be obtained

C->>G: Obtain Bearer token

G->>C: 200 OK

C-->>R: Push/pull requests

Note right of C: Bearer token included in the Authorization header

For more details, refer to the Docker documentation.

Push and Pull

Push and pull commands are used to upload and download images, more precisely manifests and blobs. The push/pull flow is described in the documentation.

GitLab Rails

GitLab Rails interacts with the registry through the HTTP API and consumes its webhook notifications.

From GitLab Rails to Registry

The single endpoint for the registry is the HTTP API. GitLab Rails invokes the API to perform all operations, which include:

Operation	Background	Observations	UI	
Check API version	{check-circle} Yes	{check-circle} Yes	Used globally to ensure that the registry supports the Docker Distribution V2 API, as well as for identifying whether GitLab Rails is talking to the GitLab container registry or a third-party one (used to toggle features only available in the former).	
List repository tags	{check-circle} Yes	{check-circle} Yes	Used to list and show tags in the UI. Used to list tags in the background for cleanup policies and Geo replication.	
Check if manifest exists	{check-circle} Yes	{dotted-circle} No	Used to get the digest of a manifest by tag. This is then used to pull the manifest and show the tag details in the UI.	
Pull manifest	{check-circle} Yes	{dotted-circle} No	Used to show the image size and the manifest digest in the tag details UI.	
Pull blob	{check-circle} Yes	{dotted-circle} No	Used to show the configuration digest and the creation date in the tag details UI.	
Delete tag	{check-circle} Yes	{check-circle} Yes	Used to delete a tag from the UI and in background (cleanup policies).	

A valid authentication token is generated in GitLab Rails and embedded in all these requests before sending them to the registry.

From Registry to GitLab Rails

The registry supports webhook notifications to notify external applications when an event occurs, such as an image push.

For GitLab, the registry is currently configured to deliver notifications for image push events to the GitLab Rails API. These notifications are currently used for Snowplow metrics and Geo replication.

Challenges

Garbage Collection

The container registry relies on an offline mark and sweep garbage collection (GC) algorithm. To run it, the registry needs to be either shutdown or set to read-only, remaining like that during the whole GC run.

During the mark phase, the registry analyzes all repositories, creating a list of configurations, layers, and manifests that are referenced/linked in each one of them. The registry will then list all existing configurations, layers, and manifests (stored centrally) and obtain a list of those that are not referenced/linked in any repository. This is the list of blobs eligible for deletion.

With the output from the mark phase in hand, the registry starts the sweep phase, where it will loop over all blobs identified as eligible for deletion and delete them from the storage backend, one by one.

Doing this for a huge registry may require multiple hours/days to complete, during which the registry must remain in read-only mode. This is not feasible for platforms with tight availability requirements, such as GitLab.com.

This limitation is also described in the upstream Docker Distribution documentation.

Performance

Due to the current architecture and its reliance on the (possibly remote) storage backend to store repository and image metadata, even the most basic operations, such as listing repositories or tags, can become prohibitively slow, and it only gets worse as the registry grows in size.

For example, to be able to tell which repositories exist, the registry has to walk through all folders in the storage backend and identify repositories in them. Only when all folders that exist have been visited, the registry can then reply to the client with the list of repositories. If using a remote storage backend (such as GCS or S3), performance becomes even worse, as for each visited folder multiple HTTP requests are required to list and inspect their contents.

Consistency

Some storage backends, like S3, can only provide eventual consistency. As an example, reading a blob after deleting it can succeed for a short amount of time. Eventual consistency could be problematic when paired with online garbage collection, as read requests at the API level could continue to succeed for deleted blobs.

Insights

For similar reasons as highlighted above, currently, it's not feasible to extract valuable information from the registry, such as how much space a repository is using, which repositories are using the most space, which ones are more active, detailed push/pull metrics for each image or tag, and others. Not having access to these insights and metrics strongly weakens the ability to make informed decisions in regards to the product strategy.

Additional Features

Due to the metadata limitations, it's currently not feasible to implement valuable features such as pagination, filtering and sorting for HTTP API, and more advanced features such as the ability to distinguish between Docker and Helm charts images.

Because of all these constraints, we decided to freeze the development of new features until we have a solution in place to overcome all these foundational limitations.

New Architecture

To overcome all challenges described above, we started an effort to migrate the registry metadata (the list of blobs, repositories, and which manifest/layers are referenced/linked in each one of them) from the storage backend into a PostgreSQL database.

The ultimate goal of the new architecture is to enable online garbage collection (&2313), but once the database is in place, we will also be able to implement all features that have been blocked by the metadata limitations. The performance of the existing API should drastically increase as well.

The introduction of a database will affect the registry architecture, as we will have one more component involved:

```
mermaid
graph LR
    C((Client)) -- HTTP request --> R(Container Registry)
    R -- Upload/download blobs --> B(Storage Backend)
    R -- Write/read metadata --> D[(Database)]
```

With a database in place, the registry will no longer use the storage backend to write and read metadata. Instead, metadata will be stored and manipulated on the PostgreSQL database. The storage backend will then be used only for uploading and downloading blobs.

The interaction between the registry and its clients, including GitLab Rails and the Docker client, will remain unchanged, as documented in the Current Architecture section. The architectural changes and the database are internal only. The registry HTTP API and webhook notifications will also remain unchanged.

Database

Following the GitLab Go standards and style guidelines, no ORM is used to manage the database, only the database/sql package from the Go standard library, a PostgreSQL driver (lib/pq) and raw SQL queries, over a TCP connection pool.

The design and development of the registry database adhere to the GitLab database guidelines. Being a Go application, the required tooling to support the database will have to be developed, such as for running database migrations.

Running online and post deployment migrations is already supported by the registry CLI, as

described in the documentation.

Partitioning

The registry database will be partitioned from start to achieve greater performance (by limiting the amount of data to act upon and enable parallel execution), easier maintenance (by splitting tables and indexes into smaller units), and high availability (with partition independence). By partitioning the database from start we can also facilitate a sharding implementation later on if necessary.

Although blobs are shared across repositories, manifest and tag metadata are scoped by repository. This is also visible at the API level, where all write and read requests (except listing repositories) are scoped by repository, with its namespace being part of the request URI. For this reason, after identifying access patterns, we decided to partition manifests and tags by repository and blobs by digest, ensuring that lookups are always performed by partition key for optimal performance. The initial version of the partitioned schema was documented in a merge request.

GitLab.com

Due to scale, performance and isolation concerns, for GitLab.com the registry database will be on a separate dedicated PostgreSQL cluster. See 93 and GitLab-com/gl-infra/reliability10109 for additional context.

The diagram below illustrates the architecture of the database cluster:

Expected Rate and Size Requirements

Rate and size requirements for the GitLab.com database were extrapolated based on the dev.gitlab.org registry and are available in the linked issues.

Self-managed instances

By default, for self-managed instances, the registry will have a separate logical database in the same PostgreSQL instance/cluster as the GitLab database. However, it will be possible to configure the registry to use a separate instance/cluster if needed.

PostgreSQL

During the discussion of the initial database schema, we have decided to opt for PostgreSQL over other database engines mainly because:

- It offers all the features we need, including the ACID guarantees of an RDBMS, and partitioning;
- It is already used for GitLab, thus we have the required experience and tools in place to manage it;
- We want to offer self-managed customers the possibility of hosting the registry database within the same PostgreSQL instance that they already have for GitLab.

Version 12

PostgreSQL introduced significant improvements for partitioning in version 12, among which we highlight:

- It's now possible for foreign keys to reference partitioned tables. This is a hard requirement for this project not only to guarantee consistency and integrity but also to enable cascading deletes at the database level;
- Major performance improvements for inserts, selects, and updates with less locking and consistent performance for a large number of partitions (benchmarks);
- Major improvements to the planning algorithm for tables with a large number of partitions, with some tests finding speedups of up to 10,000 times (source);
- Attaching new partitions to an existing table no longer requires locking the entire table;
- Bulk load (COPY) now uses bulk inserts instead of inserting one row at a time;

To leverage these features and performance improvements, we need to use PostgreSQL 12 from the start.

GitLab 14.0 and later ships with PostgreSQL 12

for self-managed instances. Customers not able to upgrade to PostgreSQL 12 have two options:

- Administrators can manually provision and configure a separate PostgreSQL 12 database for the registry. This lets you benefit from the features provided by the new registry and its metadata database.
- If online garbage collection isn't a concern or provisioning a separate database isn't possible, continue to use the current registry without the database. GitLab supports the current version with security backports and bug fixes.

Apart from online garbage collection, the metadata database's availability unblocks the implementation of many requested features for the GitLab container registry. These features are only available for instances using the new version backed by the metadata database.

Availability

Apart from the authentication service and the blob storage backend, the registry gains one more dependency with the new architecture - the database. Given that the registry can only be as reliable as its dependencies, the database deployment should be projected for high availability. A gradual migration approach should help to identify and mitigate implementation and resource constraints for high availability environments.

HTTP API

This is a list of all the registry HTTP API operations and how they depend on the storage backend and the new database. The registry will return an error response if any dependencies are not available when processing a request.

| Operation

Method	Path	Requires Database	Requires Storage	Used by GitLab Rails
----- ----- ----- -----				
----- ----- -----				
Check API version	GET /v2/		{dotted-circle}	
No	{dotted-circle} No	{check-circle} Yes		
List repositories	GET /v2/catalog		{check-circle} Yes	
{dotted-circle} No	{dotted-circle} No			
List repository tags	GET /v2/<name>/tags/list		{check-circle}	
Yes	{dotted-circle} No	{check-circle} Yes		
Delete tag	DELETE /v2/<name>/manifests/<reference>		{check-circle} Yes	
{dotted-circle} No	{check-circle} Yes			
Check if manifest exists	HEAD /v2/<name>/manifests/<reference>		{check-circle}	
Yes	{dotted-circle} No	{check-circle} Yes		
Pull manifest	GET /v2/<name>/manifests/<reference>		{check-circle} Yes	
{dotted-circle} No	{check-circle} Yes			
Push manifest	PUT /v2/<name>/manifests/<reference>		{check-circle} Yes	
{dotted-circle} No	{dotted-circle} No			
Delete manifest	DELETE /v2/<name>/manifests/<reference>		{check-circle}	
Yes	{dotted-circle} No	{dotted-circle} No		
Check if blob exists	HEAD /v2/<name>/blobs/<digest>		{check-circle}	
Yes	{dotted-circle} No	{dotted-circle} No		
Pull blob	GET /v2/<name>/blobs/<digest>		{check-circle} Yes	
{check-circle} Yes	{check-circle} Yes	{check-circle} Yes		
Delete blob	DELETE /v2/<name>/blobs/<digest>		{check-circle}	
{check-circle} Yes	{dotted-circle} No	{dotted-circle} No		
Start blob upload	POST /v2/<name>/blobs/uploads/		{check-circle}	
Yes	{check-circle} Yes	{dotted-circle} No		
Check blob upload status	GET /v2/<name>/blobs/uploads/<uuid>		{check-circle}	
Yes	{check-circle} Yes	{dotted-circle} No		
Push blob chunk	PATCH /v2/<name>/blobs/uploads/<uuid>		{check-circle}	
Yes	{check-circle} Yes	{dotted-circle} No		
Complete blob upload	PUT /v2/<name>/blobs/uploads/<uuid>		{check-circle}	
Yes	{check-circle} Yes	{dotted-circle} No		
Cancel blob upload	DELETE /v2/<name>/blobs/uploads/<uuid>		{check-circle} Yes	
{check-circle} Yes	{dotted-circle} No			

Refer to the list of interactions between registry and Rails to know why and how.

Failure Scenarios

With the addition of a database, it is essential to highlight possible failure scenarios, how we expect the registry to behave in such conditions, and their impact on the registry availability and functionality.

Database Failover

In case of refused connections or timeouts when attempting to connect to the database, which might happen during a failover scenario, the registry will discard broken connections upfront and attempt to open a new one from the pool immediately.

The application will not panic in this scenario. It will try to establish a new connection for every request. If failed, an HTTP 503 Service Unavailable error is returned to clients, and the error is logged and reported to Sentry. There is no retry cadence. The registry will only try to establish a new connection when another request that requires database access is received.

It is also possible to configure the registry to periodically check the health of the database server using a TCP health checker with a configurable interval and threshold (docs). In case the health check fails, incoming requests will be halted with an HTTP 503 Service Unavailable error.

Once the database server is available again, the registry will gracefully reconnect on the next incoming request, restoring the full API functionality without human intervention.

The expected registry behavior will be covered with integration tests, using a programmable TCP proxy between the registry and a database server to simulate a failover scenario.

Connection Pool Saturation

If unable to pull a connection from the pool to serve a given request, the registry will timeout and return an HTTP 500 Internal Server Error error to the client and report the error to Sentry. These issues should trigger a development escalation to investigate why the pool is being exhausted. There might be too much load for the preconfigured pool size, or there might be transactions holding on to connections for too long.

Prometheus metrics should be used to create alerts to act on a possible saturation before the application starts returning errors. Special attention will be paid to these scenarios during the gradual migration of the GitLab.com registry, where we will have limited, gradual, and controlled exposure on the new registry nodes. During that process, we can identify usage patterns, observe metrics, and fine tune both infrastructure and application settings accordingly as the load increases. If needed, a rate limiting algorithm may be applied to limit impact. Decisions will be based on real data to avoid overly restrictive measures and premature optimizations.

The expected registry behavior will be covered with integration tests by manipulating the pool size and spawning multiple concurrent requests against the API, putting pressure on the pool and eventually exhausting its capacity.

Latency

Excessive latency on established connections is hard to detect and debug, as these might not raise an application error or network timeout in typical circumstances but usually precede them.

For this reason, the duration of database queries used to serve HTTP API requests should be instrumented using metrics, allowing the detection of unusual variations and trigger alarms accordingly before an excessive latency becomes a timeout or service unavailability.

The expected registry behavior will be covered with integration tests, using a programmable TCP proxy between the registry and a database server to simulate increasing latency scenarios.

Problematic Migrations

Apart from unusual network and systems conditions, problematic migrations and data failures can also affect the database availability and, as a consequence, the registry availability.

Database migrations will adhere to the same development best practices used for GitLab Rails, except Rails-specific methods and tools, as the registry is a Go application with no ORM, and migrations are therefore expressed with raw SQL statements. Regardless, all changes will require a review and approval from the Database team.

Database migrations will be idempotent, with guard clauses used whenever necessary. It is also intended to guarantee that they are forward compatible. For a clustered environment, a node running registry version N should not cause any issues when the database schema is from version N+1.

Observability

Adding one more component to the system makes it even more critical to guarantee that we have proper observability over the registry's behavior and its dependencies. It should be guaranteed that we have all the necessary tools in place before rolling out the new registry backed by the metadata database.

For this purpose, error reporting with Sentry, improved structured logging, and improved HTTP metrics were already implemented and released. At the time of writing, the GitLab.com rollout is in progress.

Additionally, the Prometheus metrics will be augmented with details on the database connection pool. These will be added to the registry Grafana dashboards, joining the existing HTTP API, deployment, and storage metrics. The database cluster for the registry will also have metrics and alerts.

Together, these resources should provide an adequate level of insight into the registry's performance and behavior.

New Features and Breaking Changes

Third-Party Container Registries

GitLab ships with the GitLab container registry by default, but it's also compatible with

third-party registries, as long as they comply with the Docker Distribution V2 Specification, now superseded by the Open Container Initiative (OCI) Image Specification.

So far, we have tried to maintain full compatibility with third-party registries when adding new features. For example, in 12.8, we introduced a new tag delete feature to delete a single tag without deleting the underlying manifest. Because this feature is not part of the Docker or OCI specifications, we have kept the previous behavior as a fallback option to maintain compatibility with third-party registries.

However, this will likely change in the future. Apart from online garbage collection, and as described in challenges, the metadata database will unblock the implementation of many requested features for the GitLab container registry in the mid/long term. Most of these features will only be available for instances using the GitLab container registry. They are not part of the Docker Distribution or OCI specifications, neither we will be able to provide a compatible fallback option.

For this reason, any features that require the use of the GitLab container registry will be disabled if using a third-party registry, for as long as third-party registries continue to be supported.

Synchronizing Changes With GitLab Rails

Currently, the GitLab Rails and GitLab container registry releases and deployments have been fully independent, as we have not introduced any new API features or breaking changes, apart from the described tag delete feature.

The registry will remain independent from GitLab Rails changes, but in the mid/long term, the implementation of new features or breaking changes will imply a corresponding change in GitLab Rails, so the latter will depend on a specific minimum version of the registry.

For example, to track the size of each repository, we may extend the metadata database to store that information and then propagate it to GitLab Rails by extending the HTTP API that it consumes. In GitLab Rails, this new information would likely be stored in its database and processed to offer a new feature at the UI/API level.

This kind of changes will require a synchronization between the GitLab Rails and the GitLab container registry releases and deployments, as the former will depend on a specific version of the latter.

Feature Toggling

All GitLab Rails features dependent on a specific version of the registry should be guarded by validating the registry vendor and version.

This is already done to determine whether a tag should be deleted using the new tag delete feature (only available in the GitLab container registry v2.8.1+) or the old method. In this case, GitLab Rails sends an OPTIONS request to the registry tag route to determine whether the DELETE method is supported or not.

Alternatively, and as the universal long-term solution, we need to determine the registry

vendor, version, and supported features (the last two are only applicable if the vendor is GitLab) and persist it in the GitLab Rails database. This information can then be used in real time to toggle features or fallback to alternative methods, if possible. The initial implementation of this approach was introduced as part of 204839. Currently, it's only used for metrics purposes. Further improvements are required to guarantee that the version information is kept up to date in self-managed instances, where the registry may be hot swapped.

Release and Deployment

As described above, feature toggling offers a last line of defense against desynchronized releases and deployments, ensuring that GitLab Rails remains functional in case the registry version that supports new features is not yet available.

However, the release and deployment of GitLab Rails and the GitLab container registry should be synchronized to avoid any delays. Contrary to GitLab Rails, the registry release and deployment are manual processes, so special attention must be paid by maintainers to ensure that the GitLab Rails changes are only released and deployed after the corresponding registry changes.

As a solution to strengthen this process, a file can be added to the GitLab Rails codebase, containing the minimum required version of the registry. This file should be updated with every change that depends on a specific version of the registry. It should also be considered when releasing and deploying GitLab Rails, ensuring that the pipeline only goes through once the specified minimum required registry version is deployed.

Iterations

1. Design metadata database schema;
1. Add support for managing metadata using the database;
1. Design plans and tools to facilitate the migration of small, medium and large repositories;
1. Implement online garbage collection;
1. Create database clusters in staging and production for GitLab.com;
1. Create automated deployment pipeline for GitLab.com;
1. Deployment and gradual migration of the existing registry for GitLab.com;
1. Rollout support for the metadata database to self-managed installs.

A more detailed list of all tasks, as well as periodic progress updates can be found in the epic &2313.

Relevant Links

- Allow administrators to run garbage collection with zero downtime
- Proposal for continuous, on-demand online garbage collection
- Gradual migration proposal for the GitLab.com container registry
- Create a self-serve registry deployment
- Database cluster for container registry

SECTION: engineering/architecture/design-
documents/container_registry_metadata_database_self_managed_rollout/_index.md

{{< engineering/design-document-header >}}

Summary

The latest iteration of the container registry has been rearchitected to use a PostgreSQL database and deployed on GitLab.com. Now we must bring the advantages provided by the database to self-managed users. While the container registry retains the capacity to run without the new database, many new and highly desired features cannot be implemented without it. Additionally, unifying the registry used for GitLab.com and for self-managed allows us to provide a cohesive user experience and reduces the burden associated with maintaining the old registry implementation. To accomplish this, we plan to eventually require all self-managed to migrate to the new registry database, so that we may deprecate and remove support for the old object storage metadata subsystem.

This document seeks to describe how we may use the proven core migration functionality, which was used to migrate millions of container images on GitLab.com, to enable self-managed users to enjoy the benefits of the metadata database.

Motivation

Enabling self-managed users to migrate to the new metadata database allows these users to take advantage of the new features that require the database. Additionally, the greater adoption of the database allows the container registry team to focus our knowledge and capacity, and will eventually allow us to fully remove the old registry metadata subsystem, greatly improving the maintainability and stability of the container registr